

EUROGRAPHICS '85

Proceedings of the European Graphics Conference and Exhibition Nice, France 9-13 September, 1985

edited by

Carlo E. VANDONI

CERN

Geneva, Switzerland

PARTICIPANTS EDITION



1985

NORTH-HOLLAND

EUROGRAPHICS '85
C.E. Vandoni (Editor)
Elsevier Science Publishers B.V. (North-Holland)
© Eurographics Association, 1985

365

A NORMAL-BUFFER VECTORIZED SURFACE SHADING MODEL

Ian J. Curington

Floating Point Systems Ltd.
Bracknell, United Kingdom

A surface rendering implementation is described that separates visibility operations from surface shading operations with a Normal-Buffer data structure. This allows for scene complexity independent shading operations at a fixed computational cost per frame. The Normal-Buffer approach also allows the implementation of a fully vectorized surface-shading algorithm when the shading model is applied to each pixel in the scene. The vectorized shader is described using an array processor with a directly attached frame store in a high performance scene-generation system.

INTRODUCTION

The process of image synthesis accepts three dimensional geometric objects, such as polygons or algebraic surfaces, and displays them from a specific viewing position. In the transition from geometry to image illumination from one or more light sources is combined with local surface characteristics to determine the visible shade. Surface visibility calculations process geometric information to determine which parts of the scene are visible from the viewpoint. Shading algorithms attempt to model the physical properties of light propagation, taking into account the light source position, the surface orientation, the viewer's position and the colour and reflectivity of each surface.

Separating the visibility and shading algorithms with a surface-characteristic buffer allows the shading algorithm to operate efficiently on an entire scan line in one pass. With this linkage between the rendering steps, the shading computational cost becomes a function of display resolution and is entirely independent of scene complexity. Experience with such a system indicates a low cost of five to ten seconds for the shading algorithm compared with several minutes on a supermini computer, dependent only on the complexity of the shading model and not the number of visible objects.

The formation of the Normal-Buffer during the hidden surface process has also been vectorized for an array processor, in addition to reflections and anti-aliasing. The techniques used in the vectorized surface shader are described in detail to illustrate the performance gain.

SCAN LINE SHADING METHODS

The traditional implementations of surface-rendering programs invoke the shading algorithm for either one pixel or a small horizontal span at the inner loop of a visibility algorithm [1]. In the

implementation described in this paper, the surface visibility calculations are performed first and then a separate surface-shading algorithm is applied over one scan line. The surface visibility calculation is a simple scan-line depth buffer algorithm. A scan line algorithm makes efficient use of memory especially at high resolutions since it is not necessary to store an entire image. Also, one scan line is a convenient block size for I/O operations. Further, geometric optimizations are possible when solving visibility calculations one scan line at a time [2]. The depth buffer approach is not as efficient as other algorithms, but is simple to implement. By utilizing the horizontal span of each object across on scan line as a one dimensional array, the hidden surface calculation and Normal-Buffer construction are vectorized for an array processor. The structure of the visibility algorithm used to construct the surface shader is shown in Figure 1.

In the full implementation, a list of objects likely to intersect the current scan line is maintained to avoid traversing the entire object list at every scan line [3]. The list of active spheres is updated by measuring the distance between the y coordinate of the sphere's center and the y coordinate of the scan line. If the distance is greater than the sphere's radius it may be ignored. The algorithm shown above is optimized for the display of spherical objects, but is very similar to a type used for polygons. To model spherical objects, the following implicit representation of a sphere is used:

$$x^2 + y^2 + z^2 - r^2 = 0$$

The three-dimensional center point and radius of each sphere is held in the object definition array. At each scan line, y becomes fixed and is substituted into the spherical equation to yield an implicit circle equation in x and z. Across the forward half of each circle, the x position (distance along the scan line) is substituted to find the depth value z at each pixel.

The results accumulated by the depth-buffer-surface-visibility algorithm include the surface characteristics and surface normal direction at each pixel. These values are held in a regular scan line data structure that allows the implementation of an efficient vectorized shader.

NORMAL BUFFER DATA STRUCTURE

The output of the visibility algorithm is a set of arrays, all of which are the length of the scan line. The Z-buffer (depth-buffer) holds the z values of the nearest surfaces to the viewer, the Object-Tag buffer contains a pointer to the object definitions for each pixel, and the N-buffer stores the surface-normal-direction vector for the object at each pixel. This is the minimum information needed by the shading algorithm to assign final colours to each pixel. Surface normal information has been stored in pixel buffers in other algorithms [4], but not used to allow vectorization. An example set of FORTRAN data structure declarations is shown below:

Input data structure to vectorized shader, 512 pixel lines:

```
REAL    XSN(512)    ! Normal-Buffer, 3 words per pixel
REAL    YSN(512)    ! x,y,z Object Surface normal vectors
REAL    ZSN(512)    ! (normalized)
REAL    ZBUF(512)    ! Single Line Depth-Buffer; z values
INTEGER JTYP(512)   ! Object-Tag, identifies visible object
```

Structures for shader output and display:

```
REAL    SNI(512)    ! Shading (intensity) factors
REAL    SRR(512)    ! Intermediate shading component vector
REAL    RBUF(512)    ! RED - Output line buffer
REAL    GBUF(512)    ! GREEN - Output line buffer
REAL    BBUF(512)    ! BLUE - Output line buffer
```

LIGHTING MODEL

The reflected intensity at a point on an object surface is considered to be composed of ambient, diffuse and specular intensity components [5].

Ambient illumination is assumed to fall uniformly in all directions, so the reflected intensity is constant and independent of either light source or viewer direction. The ambient lighting term is an additive constant which decreases the scene contrast as its value is increased.

The diffuse component is derived from the concept of light generated at a point source scattering in all directions from the object surface. The visible intensity is proportional to the surface area under a given solid angle from the light source. The diffuse component does not depend on viewer direction, and simulates the effects of a dull, non-reflective surface.

The specular component is greatest when the viewer direction is at the reflection angle of the light source and surface orientation. At this point the light is reflected from the surface as a perfect mirror. The specular term decreases sharply as the viewer moves away from the reflected light direction.

In the implementation discussed below, the lighting model is a formulation by Phong [6] combining the three components to derive surface intensity. The intensity at a point on an object surface is

$$I = (\text{amb}) + (\text{diff}) (N \cdot L) + (\text{spec}) (N \cdot H)^n$$

where L is the light source direction, N is the normal to the surface and H is a vector half-way between L and the viewer direction, shown in figure 2. The exponent n is a shaping constant set to 64 in this implementation. The constants (amb, diff and spec) are used to control the surface qualities in the image.

A more advanced and realistic surface shader implementation based on the Torrance-Sparrow model [7] differs from the above Phong model by employing a more accurate approximation to the physical light and surface interactions. This model assumes the surface is composed of V grooves of mirrors oriented in random directions. Real materials can be simulated by using surface roughness and the index of refraction values. By using the Fresnel diffraction

falling on to the surface at grazing angles of incidence is far more realistically treated than in the Phong model yet is only twice as computationally expensive. The Phong model implementation is described below due to its comparative simplicity.

VECTORIZED SURFACE SHADER

The shading algorithm shown in Figure 3 derives the final shade from the surface normal orientation, the surface colour characteristics, the light source direction and the surface reflectance. These values are used in the above lighting model to compute the final surface shade. Figure 3 outlines the operations performed by the shader in simplified form, as a FORTRAN code segment.

Vector operations are generally independent of the data they operate on and perform similar operations over multiple data elements. Often, in numerical programs, loops perform repeated operations where the repetitions are independent from one another. A vector computer, such as an array processor, makes efficient use of such program constructs [8]. A vectorized program is one where the operations have been arranged to allow these high efficiency vector operations. In the example shown in Figure 4, calls are made to a vector library that takes advantage of pipelining and multiple data paths within the array processor.

The example in Figure 4 is vectorized by use of the vector math library on an FPS-5205 array processor [9]. This implementation is shown in FORTRAN for scan lines of length 512 pixels.

This structure is enhanced to model multiple coloured-light sources by repeating the shader for several light source directions and combining the individual intensities before merging the final colours [10,11]. Because the shading model is independent of the hidden surface process multiple light sources are a fixed additional computational cost per image.

COMPUTER SYSTEM

The computer system uses a host computer for program development and process supervision while the array processor performs both the computationally intensive portions of the program and the display-list processing for the graphic display. The high performance graphic display system is connected directly to the array processor. Since most of the graphical data is generated by the array processor, much higher speed communication than using the host computer as the communication path is achieved. The effective bandwidth between the array processor and the frame store memory is 9 Megabytes per second.

A VAX 11/780 running VMS provides general services, including a FORTRAN-77 cross compiler for the array processor. The FPS-5205 array processor is the primary computational element in the system, and contains a DMA interface to the VAX, 256K words of data memory (38-bit), 8K words of program memory, and a GPIOP programmable interface. The GPIOP is a bit-slice I/O processor option for the FPS-5205 that executes a graphics executive for fast operation of the graphics frame store memory. The GPIOP contains its own program store and a high speed format conversion processor.

PERFORMANCE ANALYSIS

The vectorized shader executes in a fixed time, directly proportional to display resolution in pixels and independent of scene complexity. Counting the operations of floating point addition, multiplication, and memory references, the vectorized shading model is estimated to execute at 3.8 MFLOPS (million floating point operations per second) on the FPS-5205. Practical tests show that at 512 by 512 pixel resolution, the vectorized shader takes 5.8 seconds per image.

The spherical surface shading program has been used to display scenes of different complexity with small linear growth in computation time with the number of spheres. For example, the simple spherical visibility algorithm shown above when vectorized for an array processor takes approximately 12 seconds to produce an image with 400 spheres, including rotation of the model and output to the graphic frame store. Figure 5 shows the execution time for scenes with a variable number of spheres displayed at three different resolutions. The hidden surface process has linear cost with the number of spheres, with small variations due to varying depth complexity and visible foreground to background ratio. Figure 4 shows times for one white Phong shading model, with no anti-aliasing. The I/O transfer time for sending the final pixel values to the display is fully overlapped with the processing time by using the programmable I/O processor (GPIOP).

EXAMPLES

The surface shading system is used for several applications, including molecular graphics and a particle accretion modelling system. All of the plates were produced at a display resolution of 512 by 512 pixels. The computational resolution was varied from 512 by 512 to 1024 by 1024 pixels to test the algorithm performance. At 512 by 512 resolution, the Phong surface-shading calculation takes 5.8 seconds, while the Torrance-Sparrow calculation takes 12.2 seconds, independent of the number of objects in the image.

Plate 1* shows an Analine molecule, composed of 13 atoms, computed at 512 by 512 resolution. Part of a molecular graphics program, the time required to produce this view is 8 seconds without any form of anti-aliasing. The atoms appear elliptical due to a rectangular aspect ratio video monitor.

Plate 2* is of a procedural warped surface composed of 2500 spheres. The spheres are coloured according to their radius and shaded using the Torrance-Sparrow specular reflection model. This image was computed at 512 by 512 pixel resolution using anti-aliasing and 2 directional coloured light sources. The time to produce Plate 2 was 93 seconds.

Plate 3* is a model of cosmic dust created by a computer simulation of the accretion of particle aggregates [12]. A statistical process models the growth of particle aggregates by the accretion of smaller particles colliding in random trajectories. The colours are assigned to each sphere according to its radius. The aggregate shown in Plate 3 is composed of 5000 individual spheres. The time to generate the image at 512 by 512 resolution with one Phong light source and anti-aliasing was 112 seconds.

DISCUSSION

The methods described in this paper are useful for high speed implementation when a complex shading model can be applied to the entire scene, regardless of complexity or object distributions, with values determined at pixel resolution. The extensions to more elaborate rendering techniques involving object dependent algorithms, or analytic anti-aliasing techniques are more difficult.

The vectorized shader poses special problems with anti-aliasing [13]. The regular data structure will not directly extend to adaptive subdivision or analytical anti-aliasing such as area sampling and still be easily vectorized. The method used in the present implementation is to use a high computational pixel resolution and then use a decimating convolution filter to resample at display resolution. This technique is not a general solution to aliasing but removes the most obvious stair-step effects.

The traditional methods of rendering surfaces, particularly with simple shading models, apply shading calculations to relatively few pixels in the scene. Key colours, from shades determined at vertices of polygons, may be used to fill a region, or linearly interpolated across regions. The shading model shown in this paper is useful when the shading model is applied separately to all pixels in the scene utilizing differences between surface normals in adjacent pixels.

This shading model also assumes that the same model is appropriate for all surfaces in a scene. If different shading models are required for different objects the vectorization techniques shown here are much harder to implement. However, using the Normal-Buffer may still be helpful in isolating the computation time from scene complexity.

Recent investigations have led to an extension of the Normal-Buffer vectorization technique for ray tracing. The surface normal information is expanded to also include the viewer direction vector, which is used for a scan-line calculation of shadows from multiple light sources, and reflected virtual images. By using the viewer rays instead of a fixed screen geometry, the same vectorized shader described above is applied at each level of ray recursion.

SUMMARY

Application of a complex but uniform shading model allows separation from the visibility algorithm with a surface Normal-Buffer and allows the efficient vectorization of the shading model. An efficient implementation of this algorithm is shown for a high speed Array Processor. This technique provides a solution when complex shading algorithms require large amounts of floating point computation. This paper demonstrates how vector architectures are well suited to realistic surface-shading models when a Normal-Buffer algorithm is employed.

REFERENCES

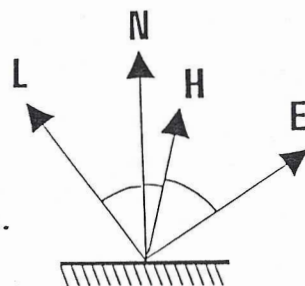
- [1] Foley and Van Dam, "Fundamentals of Interactive Computer Graphics" Addison Wesley publisher, 1982.
- [2] Whitted, Turner "Processing Requirements for Hidden Surface Elimination and Realistic Shading" Digest of Papers, COMPCON Spring '82, IEEE 1982.
- [3] Porter, Thomas "Spherical Shading" Computer Graphics, Vol. 12, No. 3, 1978.
- [4] Brotman, L. S., and Badler, N. I. "Generating Soft Shadows with a Depth Buffer Algorithm" IEEE Computer Graphics and Applications, Vol. 4, No. 10, October 1984.
- [5] Whitted, Turner "An Improved Illumination Model for Shaded Display" Communications of the ACM, Vol. 23, No. 6, June 1980.
- [6] Phong, Bui Toung "Illumination for Computer Generated Pictures" Communications of the ACM, Vol. 18, No. 6, pp. 311-317, 1975.
- [7] Torrance, Kenneth. E. and Sparrow, Ephraim. M. "Theory for Off-Specular Reflection From Roughened Surfaces" Journal of the Optical Society of America, Vol. 57, pp. 1105-1114, 1967.
- [8] Hockney, R. W., and Jesshope, C. R. "Parallel Computers: Architecture, Programming, and Algorithms" Adam Hilger Ltd. publisher, Bristol, 1981.
- [9] "FPS-5000 APMATH Manual D.4" Floating Point Systems technical publication 860-7437-011A, September 1983, Beaverton Oregon.
- [10] Max, Nelson "Vectorized Procedural Models for Natural Terrain: Waves and Islands in the Sunset" Computer Graphics Vol. 15, No. 3, August 1981.
- [11] Max, Nelson "Computer Representation of Molecular Surfaces" IEEE Computer Graphics and Applications, Vol 3, No 5, pp 21-29, August 1983.
- [12] Daniels, Paul A., and Curington, Ian J. "Particle Accretion Simulation with a Shaded Surface Display" Proceedings of the FUSE Conference, Munich, May 1985.
- [13] Weinberg, Richard "Parallel Processing Image Synthesis and Anti-Aliasing" Computer Graphics Vol. 15, No. 3, August 1981.

```

/* implicit spherical surface display */
for (y=top to bottom) {
    set entire Z-buffer to -infinity;
    set entire N-buffer to unity;
    set entire Object-tag buffer to background;
    for (Object 1 to Object N) {
        substitute y into sphere equation to get circle;
        in x-z plane;
        for (x=left to right edge of object) {
            substitute x into circle equation to find z;
            if (z > Z-buffer [x] ) {
                N-buffer [x] = surface normal at x,y,z;
                Object-tag [x]= Current Object;
                Z-buffer [x] = z;
            }
        }
    }
    call vectorized shader;
    send result colours to display;
}

```

Figure 1. Outline of Implicit Spherical Z-buffer Algorithm.

Figure 2.
Surface Reflection Vector Notation.
H is half way from L to E.

```

DO 100 J=1,512
SRR(J) = XSN(J)*XLN + YSH(J)*VLN + ZSN(J)*ZLN ! Normal dot light
SNI(J) = XSN(J)*XLR + YSH(J)*VLR + ZSN(J)*ZLR ! Normal dot reflect
IF (SRR(J) .LT. 0.0) SRR(J) = 0.0 ! no negative light
IF (SNI(J) .LT. 0.0) SNI(J) = 0.0 ! on shadow sides
DO 50 I=1,6
SNI(J) = SNI(J) * SNI(J) ! raise hi-lite
SRR(J) = DIFF*SRR(J) + AMB ! combine 2 light terms
SNI(J) = SPEC*SNI(J) ! derive specular term
RBUF(J) = SRR(J) * RMCOLR (J,TYP(J)) ! resolve surface colors
GBUF(J) = SRR(J) * GMCOLR (J,TYP(J)) ! with ambient and
BBUF(J) = SRR(J) * BMCOLR (J,TYP(J)) ! diffuse
RBUF(J) = (SNI(J) + RBUF(J)) * 255.0 ! add spectral term as
GBUF(J) = (SNI(J) + GBUF(J)) * 255.0 ! white light scaled
BBUF(J) = (SNI(J) + BBUF(J)) * 255.0 ! to byte range
IF (RBUF(J) .LT. 0.0) RBUF(J) = 0.0 ! clip range
IF (GBUF(J) .LT. 0.0) GBUF(J) = 0.0 ! to black and
IF (BBUF(J) .LT. 0.0) BBUF(J) = 0.0 ! white
IF (RBUF(J) .GT. 255.0) RBUF(J) = 255.0 ! for saturated
IF (GBUF(J) .GT. 255.0) GBUF(J) = 255.0 ! hi-lites
IF (BBUF(J) .GT. 255.0) BBUF(J) = 255.0
100 CONTINUE

```

Figure 3. Simplified FORTRAN Shading Algorithm.

```

CALL VSMUL (XSN,1,XLN,SRR,1,512) ! dotpr surface * light norm
CALL VDMA (YSN,1,VLN,SRR,1,512)
CALL VDMA (ZSN,1,ZLN,SRR,1,512)
C
CALL VSMUL (XSN,1,XLR,SNI,1,512) ! dotpr surface * reflection
CALL VDMA (YSN,1,VLR,SNI,1,512)
CALL VDMA (ZSN,1,ZLR,SNI,1,512)
C
CALL VCLIP (SRR,1,ZERO,ZMAX,SRR,1,512) ! set negatives to zero
CALL VCLIP (SNI,1,ZERO,ZMAX,SNI,1,512) ! (only ambient on back sides)
C
DO 100 I=1,6
CALL VSQ (SNI,1,SNI,1,512) ! raise hi-light to a power
CALL VDMA (SRR,1,DIFF,AMB,SRR,1,512) ! scale specular, term
CALL VSMUL (SNI,1,SPEC,SNI,1,512) ! merge diffuse, ambient terms
C
CALL VINDE (RMCOLR,JTYP,1,RBUF,1,512) ! resolve object tags to colors
CALL VINDE (GMCOLR,JTYP,1,GBUF,1,512) ! 3 to 1 data expansion
CALL VINDE (BMCOLR,JTYP,1,BBUF,1,512) ! into separate color components
CALL VMA (RBUF,1,SRR,1,SNI,1,RBUF,1,512) ! multiply surface color by
CALL VMA (GBUF,1,SRR,1,SNI,1,GBUF,1,512) ! the shader illumination
CALL VMA (BBUF,1,SRR,1,SNI,1,BBUF,1,512) ! adding specular term
C
CALL VCLIP (RBUF,1,ZERO,ONE,RBUF,1,3*512) ! clip for white saturation
CALL VSMUL (RBUF,1,255.0,RBUF,1,3*512) ! scale to byte range

```

Figure 4. Vectorized Shading Algorithm.

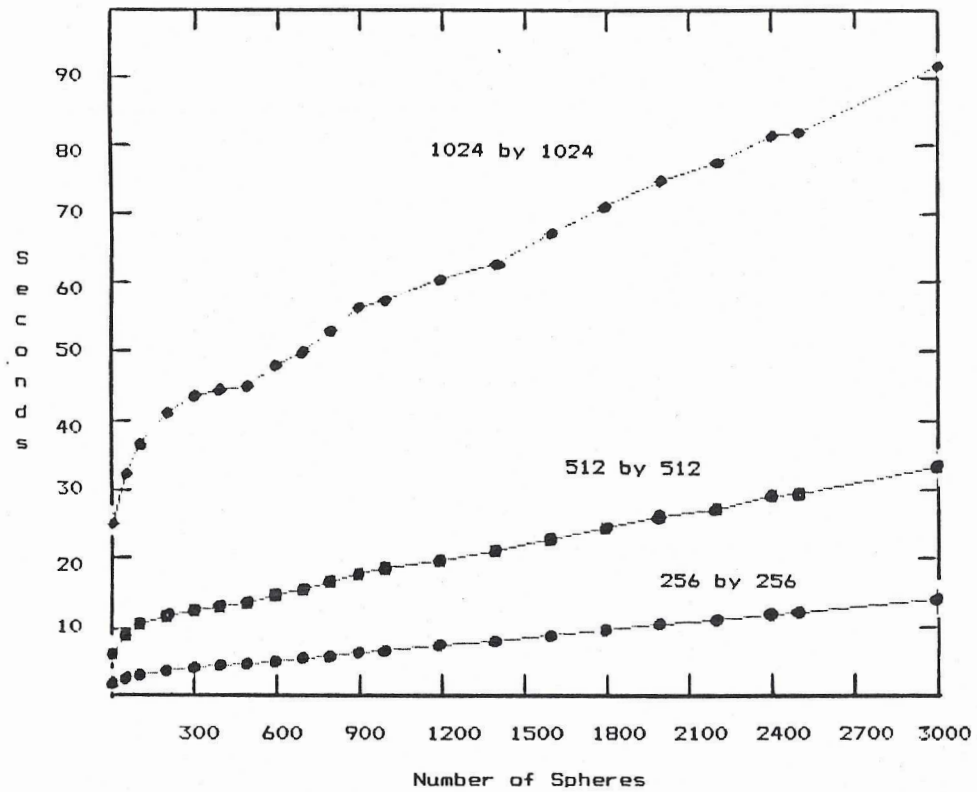


Figure 5. Execution times for scenes with increasing number of spheres at various pixel resolutions. Each time is for a scene with one Phong light and no anti-aliasing techniques.

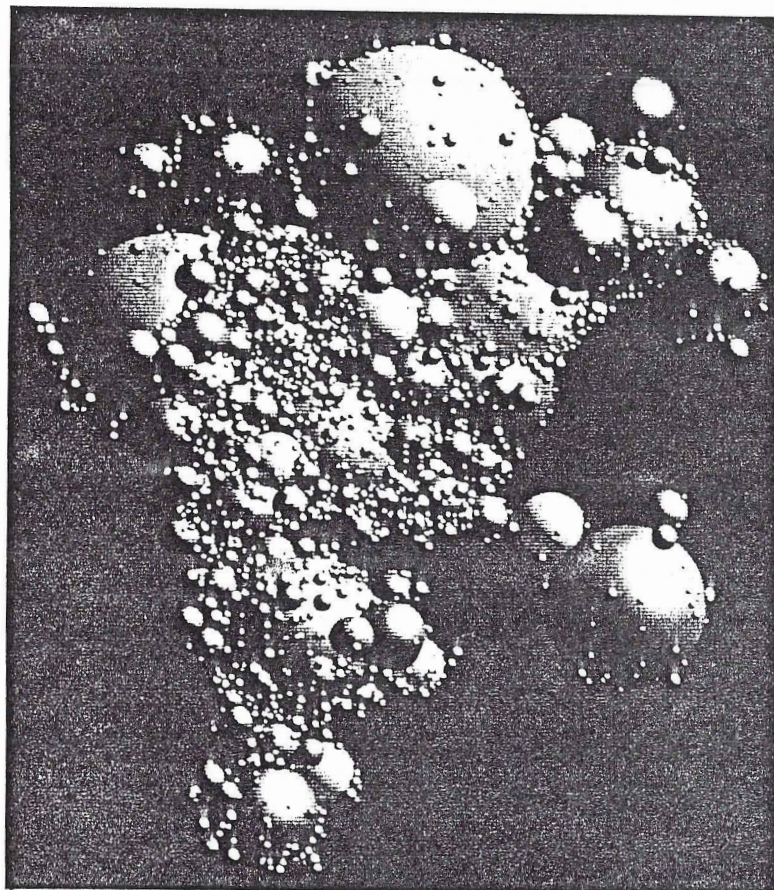


Plate 3. Simulated Cosmic Dust Particle composed of 5000 spheres. A simulation of a particle accretion process builds a large cosmic dust particle as surface contact spheres. One directional light source, Phong shading, and anti-aliasing are used in the image generation.

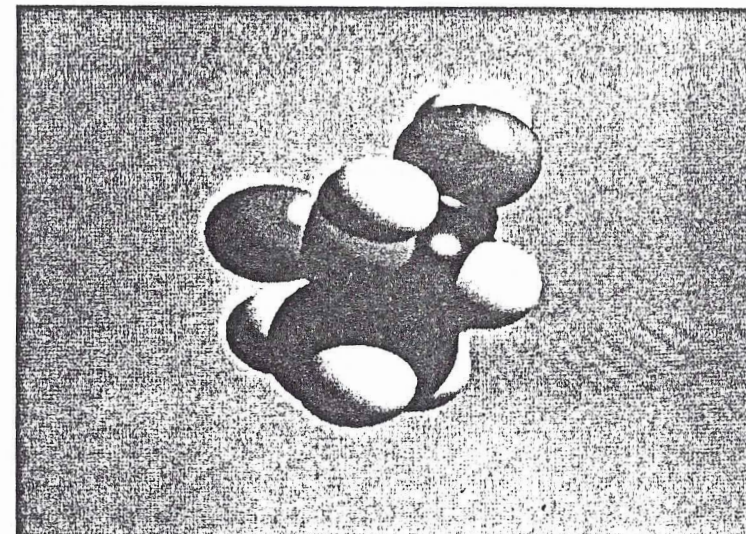


Plate 1. Aniline Molecule, 13 Atoms, at 512 by 512 resolution with no anti-aliasing. A molecular graphics program generates views of molecules with one Phong light source in 7 seconds using an FPS-5205.

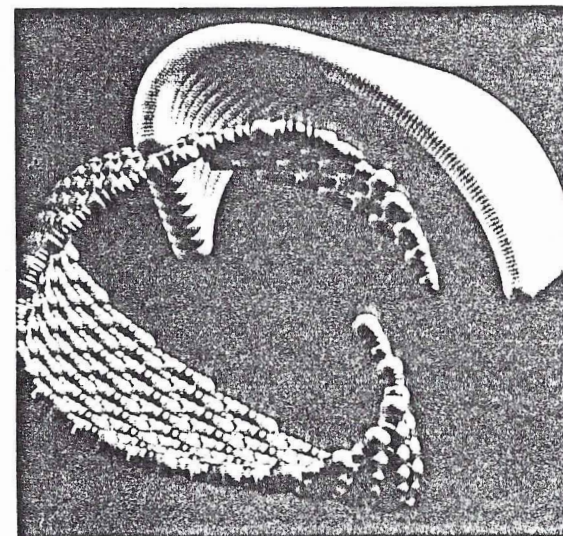


Plate 2. Warped Surface Composed of 2500 Spheres. Spheres are centered at the grid points of two second order surfaces. The radii are set to large values to simulate tubes. Torrance-Sparrow shading, several light sources, and anti-aliasing are used to create the image in 93 seconds.

